RAD 2.0 Writeup - Team Rudra_Root



Team: Rudra_Root

WEB

1. Echo Beyond the Horizon

Points: 100

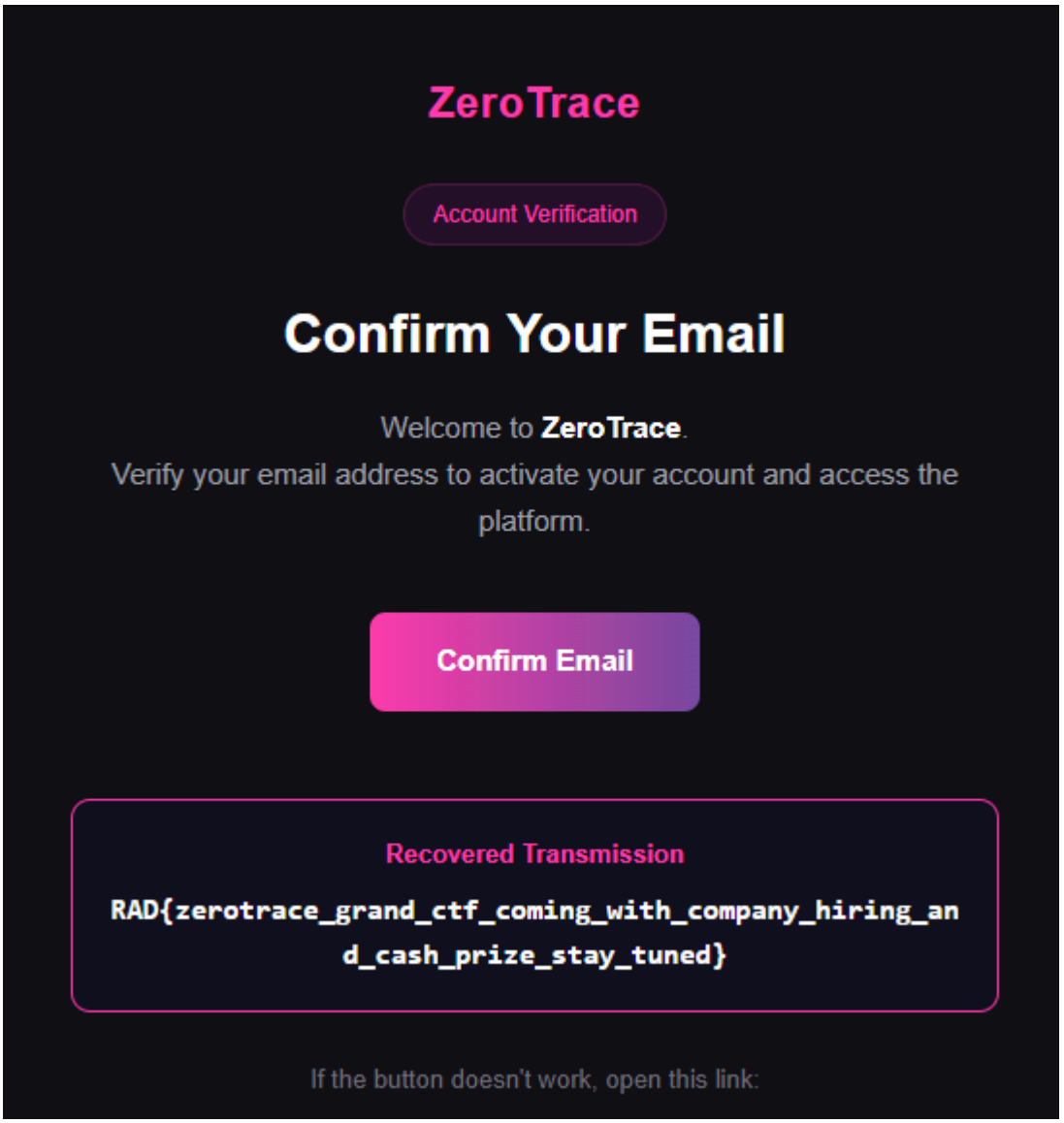
📝 Description

> For decades, humanity listened to the stars, waiting for something to answer back. Recently, a faint signal appeared — not from deep space, but buried within the digital noise of Earth's networks. The transmission is fragmented, scattered across public channels as if someone deliberately left a trail. Investigators tracing the anomaly discovered the first fragment hidden within the broadcasts of the organizing mission. Observers claim that interacting with the signal — specifically by transmitting a response containing the keyword “zerotrace” — triggers an automated relay. That relay points toward an archive maintained by "ZeroTrace", a repository believed to store remnants of the original transmission. The archive is publicly accessible, but its systems are selective. "Only explorers who establish communication through the correct channel will receive the final transmission." Some gateways may appear convenient. Some authentication routes may seem faster. But the archive only responds to the intended signal path. Follow the fragments. Trigger the relay. Access the archive. Somewhere beyond the noise… the message is still waiting. Flag Format: RAD{...} dont use any third party auth and then check email

🔍 Solution

The description strongly hints at avoiding third-party logins (like Google SSO) and using a direct registration route instead.

1. Go to **zerotrace.in**.
2. Create a new account manually using your email address (do not use third-party auth).
3. Check your email inbox. The registration confirmation email contains the hidden message and the flag.



🚩 The Flag

`RAD{zerotrace_grand_ctf_coming_with_company_hiring_and_cash_prize_stay_tuned}`

2. Event Horizon Authentication

Points: 150

📝 Description

> The Endurance station has been drifting dangerously close to the Event Horizon of a black hole.
> Something unusual has happened to its authentication system.
> To prevent intrusions, the station constantly recalculates and rewrites its access credentials every few seconds. The interface refreshes repeatedly, making it almost impossible to manually enter the correct username and password before they expire.
> However, fragments of the current credentials are leaking through the station's surface telemetry logs.
> There's just one problem.
> The system reinitializes every 3 seconds.

> If you try entering the credentials manually, the interface will likely refresh before the login completes.
> A final message from the station engineers appears in the logs:
> “Time behaves differently near the horizon. If you want access, you’ll have to stop time… or move faster than it.”
> Your objective is to obtain valid credentials and gain access to the station.
> ---> http://34.131.85.155:5001/ <---

🔍 Solution

Initial Observation

Opening the challenge site reveals the **ENDURANCE AUTH TERMINAL**. UI. It has a username and password field, a countdown bar labeled **CREDENTIAL EXPIRY**, and an **AUTHENTICATE** button.

Within 3 seconds, the entire page reloads and the credentials rotate. Trying to type manually is hopeless — the page refreshes before you can finish.

The status block gives an early hint:

```
● ROTATING ACCESS KEYS ..... ENABLED
● CREDENTIAL CYCLE ........... 3.0s
```

Source Code Analysis

Viewing the page source reveals two things immediately:

1. Credentials are leaked as HTML comments:
   ```
   <!-- username: tars_4736 -->
   <!-- password: tesseract_3101 -->
   <!-- STARFIELD BACKGROUND -->
   ```
2. **2. The JavaScript has a suspicious `window.stop()` override:**

```
window.stop = function() {
  clearTimeout(reloadTimer);          // cancels the 3-second reload
  cancelAnimationFrame(animFrameId); // halts the countdown bar

  console.log("%c[SYSTEM] CREDITENTIAL ROTATION HALTED.", ...);
  // ...
};
```

The challenge description literally says *"stop time"* — and `window.stop()` is a real browser API that halts page loading. Here, it's been **overridden** to also cancel the reload timer, essentially freezing the credential rotation.

Exploit

Step 1: Open the challenge page and immediately launch DevTools (`F12` → Console tab).
Step 2: Type and execute:
`window.stop()`
The countdown bar freezes red, and the status updates to `ROTATING ACCESS KEYS ..... HALTED`. Time has been stopped.

“Pasted image 20260316223544.png” could not be found.

Step 3: Read the current credentials from the **Elements** tab (or page source) and enter the credentials in the login form and click AUTHENTICATE. You will be redirected to the flag page (dashboard).



The flag (dashboard) url was also leaked in the JS code LOL

```
      if (enteredUser === currentUsername && enteredPass ===
currentPassword) {
        // Success — redirect to dashboard
        window.location.href = 'ewgfedfgeeicuhseunchaidvdsvb.html';
      } else {
        // Show error
        errorDiv.classList.add('visible');
        setTimeout(() => errorDiv.classList.remove('visible'), 2000);
      }

      return false;
    }
```

🚩 The Flag

`RAD{3v3nt_H0r1z0n_By_P4553d}`

3. Gargantua Relay Station

Points: 300

📝 Description

> The Deep Space Signal Relay Station is responsible for transmitting encrypted communications between astronauts on distant missions.
> Each crew member accesses the system using a secure encrypted token embedded in the URL.
> You were granted temporary access as Cooper, earlier use those credentials to login the panel.
> Mission control suspects the Admin console contains restricted transmission data.
> Can you reconstruct the system's encryption key and access the admin relay dashboard?
> Flag format: RAD{...}
> -->http://34.131.215.196:8080/login<--

🔍 Solution

Initial Observation

The challenge provides a login panel for a "Deep Space Signal Relay Station." Using the provided credentials for the user "Cooper," we successfully authenticate and are redirected to a URL containing an encrypted token:
`http://34.131.215.196:8080/user/42035fa0555f7d4a2389307eaaf7de0a`

The token `42035fa0555f7d4a2389307eaaf7de0a` is exactly 32 hex characters (16 bytes), heavily implying block-cipher cryptography like AES. The goal is to figure out the encryption

scheme, decrypt Cooper's token to see the plaintext structure, and forge a new token for the `admin` user.

Source Code Analysis

Inspecting the web page assets and source code reveals comments left behind by the developers. The system requires three fragments to reconstruct the final transmission key string, which are scattered across the frontend:

1. **Fragment 1 (HTML Source):** Hidden in an HTML comment on the `/login` page (last div statement): `FRAGMENT_1_B64 = "Z3Jhdm10eV8="` Decoding from Base64 yields: `gravity_`
2. **Fragment 2 (`relay.js`):** Found in the primary comms protocol script: `const SIGNAL_FRAG2 = "dHJhbnNjZW5kc18=";` Decoding from Base64 yields: `transcends_`
3. **Fragment 3 (`console.js`):** Found in the time dilation interface script: `const TimeMod = "dGltZQ==";` Decoding from Base64 yields: `time`

Cryptographic Analysis

The scattered developer comments also perfectly detail the encryption implementation:

- **Algorithm:** AES-256-CBC (from the HTML source comments).
- **Padding:** PKCS7 (from the HTML source comments).
- **Key Derivation:** The reconstructed key string is digested via SHA256 before encryption (`relay.js`).
- **Initialization Vector (IV):** Fixed to 16 null bytes (`console.js`).

The Exploit: Forging the Admin Token

With the exact cryptographic parameters mapped out, we can write a script to generate the SHA256 key, decrypt Cooper's token to verify the plaintext format, and encrypt the `admin` string to forge our new URL token.

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
import hashlib

# 1. Reconstruct and hash the key string
key_string = b"gravity_transcends_time"
key = hashlib.sha256(key_string).digest()

# 2. Set the fixed IV (16 null bytes)
iv = b'\x00' * 16

# 3. Decrypt Cooper's token to verify the plaintext casing
cooper_token = bytes.fromhex("42035fa0555f7d4a2389307eaaf7de0a")
cipher_dec = AES.new(key, AES.MODE_CBC, iv)
```

```
cooper_plaintext = unpad(cipher_dec.decrypt(cooper_token), AES.block_size)
print(f"Decrypted Cooper Token: {cooper_plaintext.decode()}")

# 4. Forge the Admin token
# (Assuming the decrypted Cooper token is simply "cooper")
admin_plaintext = b"admin"
cipher_enc = AES.new(key, AES.MODE_CBC, iv)
admin_token = cipher_enc.encrypt(pad(admin_plaintext, AES.block_size)).hex()

print(f"Forged Admin Token: {admin_token}")
print(f"Admin URL: http://34.131.215.196:8080/user/{admin_token}")
```

Running this script successfully decrypts the payload and outputs the forged hex token. Navigating to the generated URL grants PRIORITY ONE clearance and access to the restricted transmission data.

🚩 The Flag

`RAD{k3y_sh0u1d_n07_b3_pub11c}`

WELCOME

1. Docking at Cyberzee Station

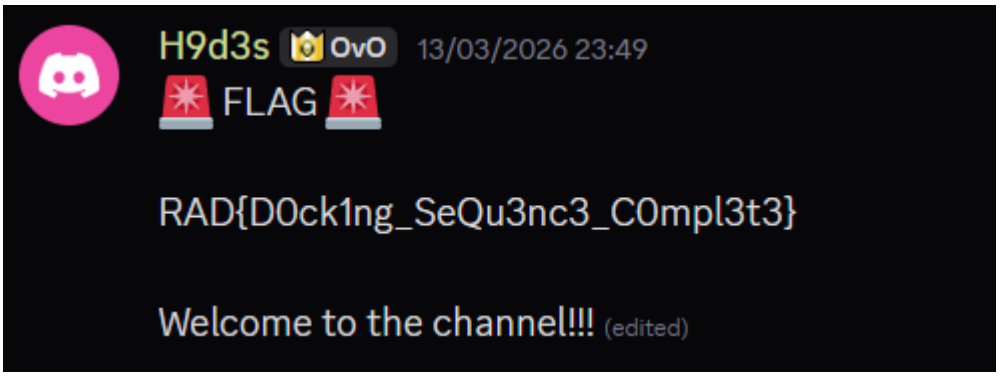
Points: 50

📝 Description

> Before any mission begins, astronauts must dock at the central station.
> Your spacecraft has reached the Cyberzee orbital hub.
> Join the Cyberzee Discord and enter the RAD sector. Once your presence is detected, Mission Control will transmit a signal directly to your personal communicator.
> Retrieve the transmission and submit the flag.
> The journey beyond the wormhole awaits.

🔍 Solution

Join the discord server and go to the `#ctf` channel to get the flag



🚩 The Flag

`RAD{D0ck1ng_SeQu3nc3_C0mpl3t3}`

2. Signal from the Deep Space Telescope

Points: 50

📝 Description

> Our deep space telescope has captured a signal from a distant galaxy.
> Navigate to the Clubs Instagram outpost and follow the station to stay connected with our transmissions.
> Mission Control has hidden a communication relay inside the event announcement broadcast transmitted on "March 5".
> To confirm that your signal has reached us, reply to that broadcast with the message: RAD2.0
> Once Mission Control detects your signal, a private transmission will be sent directly to your communicator.

🔍 Solution

- Follow the club's Instagram account: `cyberzee_srm`
- Find the CTF announcement post on March 5
- Comment 'RAD2.0' and you will get a DM with the flag.

🚩 The Flag

`RAD{C0sm1c_S1gn4l_Tr4nsM1ss10n_Cyb3rZ33_X7}`

3. The Astronaut Registry

Points: 50

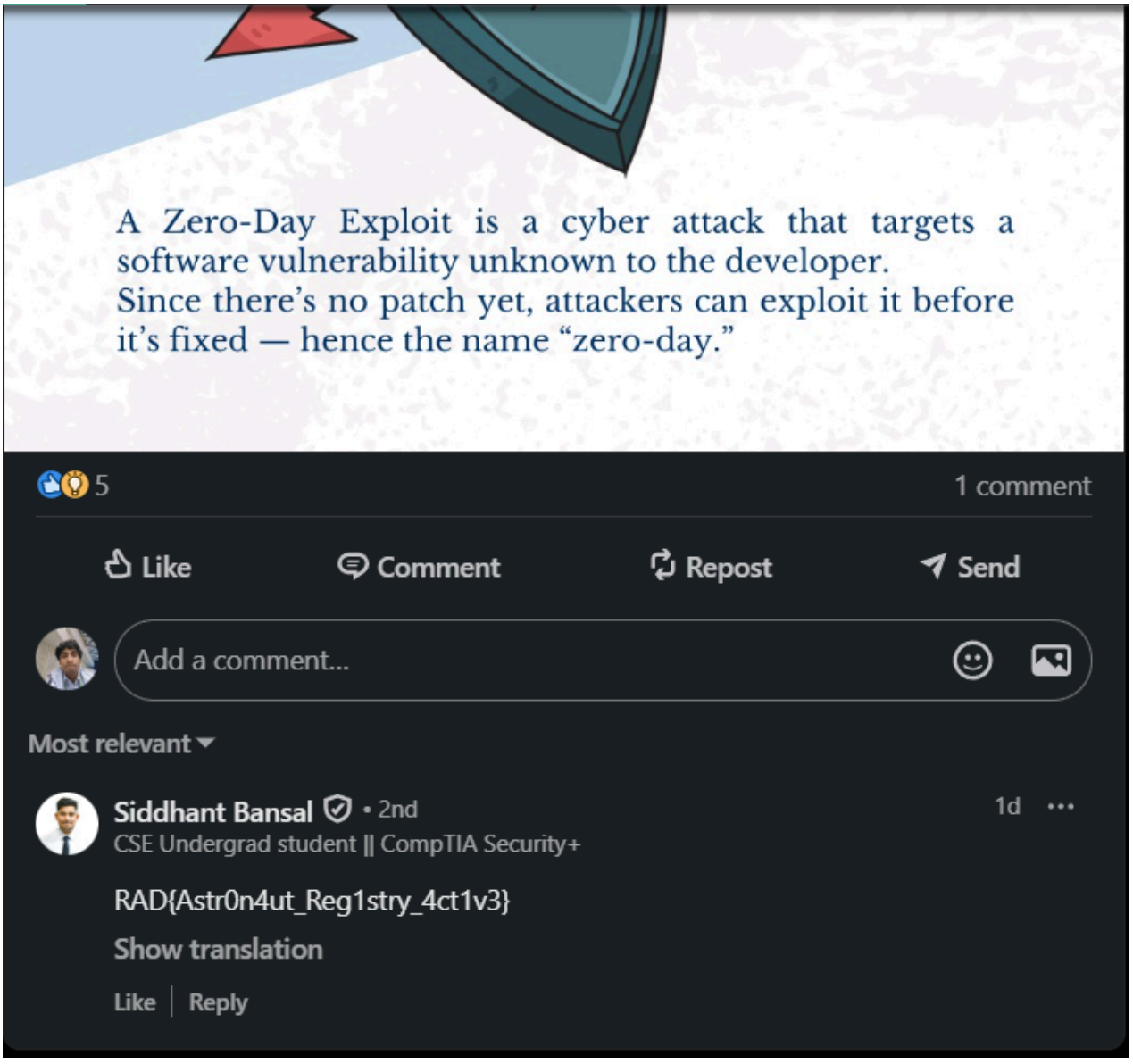
📝 Description

> Every astronaut joining the expedition must be registered with the Interstellar Mission Council.

> Locate the Cyberzee command center on LinkedIn and follow the station to confirm your presence in the registry.
> Mission Control has already transmitted a signal within the communications feed of the latest broadcast. Carefully inspect the discussion channel beneath the latest transmission.
> Retrieve the signal left by Mission Control and submit it as the flag to confirm your registration.
> -->Linkedin<--

🔍 Solution

- Find the latest post from the given LinkedIn profile and open the comments



🚩 The Flag

`RAD{Astr0n4ut_Reg1stry_4ct1v3}`

4.Wormhole Access

Points: 50

📝 Description

> You are now ready to approach the wormhole.
> Mission Control requires proof that you have successfully completed all preliminary checks across our communication stations.
> Once you have received the transmissions from Discord, Instagram and LinkedIn use the given data to activate the wormhole coordinates.
> The stars await.

🔍 Solution

Open the given data.txt file and find the flag!

🚩 The Flag

`RAD{W0rmh0l3_N4v1g4t10n_Cyb3rZ33_R4D2_2026}`

CRYPTO

1. Hidden in Plain Sight

Points: 100

📝 Description

> A strange encoded string was recovered during a quick investigation. It doesn't look like much, but sometimes the simplest layers hide the real secret. Decode the message and uncover the hidden flag.

🔍 Solution

The string in the txt file was encoded in **3 layers of Base64**:
Using CyberChef:

Layer	Result
Original	Vld0R1JXVjZUakphVmtvMVdERk5lbGw2UW5WYVJGWm1XWHBDTVdKcVl6Rr
Base64 Decode ×1	VWtGRWV6TjJaVko1WDFNell6QnVaRFZmWXpCMWJqYzFmUT09
Base64 Decode ×2	UkFEezN2ZVJ5X1MzYzBuZDVfYzB1bjc1fQ==
Base64 Decode	RAD{3veRy_S3c0nd5_c0un75}

Layer	Result
×3	

🚩 The Flag

`RAD{3veRy_S3c0nd5_c0un75}`

2. Singularity Echo

Points: 400

📝 Description

> "Cooper is somewhere inside the Tesseract, trying to communicate through gravity. TARS has analyzed the dust patterns and found a stream of coordinates, but most of it is just gravitational noise."
> **TARS Log:** "The signal is hidden. The truth only exists where the Primes uphold the Germains."
> Filter the noise from `singularity.txt`. Reconstruct the hidden curve using the valid points. The secret key lies at the origin.

🔍 Solution

This challenge is a fantastic combination of number theory and Shamir's Secret Sharing (SSS).

1. **Analyze the Hints:**
   - "Reconstruct the hidden curve... secret key lies at the origin": This is the textbook definition of Shamir's Secret Sharing, where a secret is divided into points on a polynomial curve, and the secret itself is the y-intercept (where $x = 0$).
   - "The truth only exists where the Primes uphold the Germains": We are given a massive list of coordinate pairs (noise), but we only want the ones where the x-coordinate is a **Sophie Germain Prime**. A number p is a Sophie Germain prime if both p and $2p + 1$ are prime.
2. **Extract & Filter Data:** We parse `singularity.txt` to extract the modulus, the AES encrypted payload, and the stream of coordinates. We filter the coordinates, keeping only the pairs where x is a Sophie Germain prime.
3. **Reconstruct the Secret:** Using the filtered points, we perform Lagrange Interpolation at $x = 0$ over the given finite field (using the provided modulus) to recover the constant term of the polynomial. This constant is our secret.
4. **Decrypt the Flag:** We convert the recovered secret to a string, hash it using SHA-256 to derive a 32-byte AES key, and decrypt the payload using AES-ECB with standard unpadding.

Here is the Python solver script used to automate the extraction, interpolation, and decryption:

```
import re
import hashlib
from math import isqrt
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

# Constants from singularity.txt
MOD = 340282366920938463463374607431768211507
PAYLOAD =
bytes.fromhex('b31d6447ca32f6b035fb5d458be09fe5c4ff40881892526460a9f459f0314
dc1')

def is_prime(n: int) -> bool:
    if n < 2: return False
    if n % 2 == 0: return n == 2
    for d in range(3, isqrt(n) + 1, 2):
        if n % d == 0: return False
    return True

def is_sophie_germain(n: int) -> bool:
    return is_prime(n) and is_prime(2 * n + 1)

def recover_secret(shares, mod):
    """Performs Lagrange Interpolation at x=0 to find the constant term."""
    secret = 0
    for i, (xi, yi) in enumerate(shares):
        num, den = 1, 1
        for j, (xj, _) in enumerate(shares):
            if i == j: continue
            num = (num * -xj) % mod
            den = (den * (xi - xj)) % mod

        # Multiply yi by the Lagrange basis polynomial evaluated at 0
        term = (yi * num * pow(den, -1, mod)) % mod
        secret = (secret + term) % mod
    return secret

def main():
    pts = []
    # 1. Parse and filter points
    with open('singularity.txt', 'r') as f:
        for line in f:
            match = re.fullmatch(r'(\d+),(\d+)', line.strip())
            if match:
                x, y = map(int, match.groups())
                if is_sophie_germain(x):
```

```
                    pts.append((x, y))

    # 2. Recover the secret (f(0))
    secret = recover_secret(pts, MOD)

    # 3. Derive key and decrypt
    key = hashlib.sha256(str(secret).encode()).digest()
    cipher = AES.new(key, AES.MODE_ECB)

    decrypted = cipher.decrypt(PAYLOAD)
    flag = unpad(decrypted, AES.block_size).decode()

    print(f"Recovered Secret: {secret}")
    print(f"Flag: {flag}")

if __name__ == "__main__":
    main()
```

```
└─$ python3 crypto.py
Recovered Secret: 498029797830765363198014179253508481 00
Flag: RAD{N3veR_Tru57_y0uR_D3Lu5i0n}
```

🚩 The Flag

`RAD{N3veR_Tru57_y0uR_D3Lu5i0n}`

OSINT

1. The Corns

Points: 100

📝 Description

> During the production of Interstellar, director Christopher Nolan chose to create the film's farmland setting using practical methods rather than visual effects. Find the coordinates of the town where it was planted.
>
> **Flag Format:** `RAD{ab.cd,ef.gh}`

🔍 Solution

This challenge requires finding the real-world filming location for the cornfield scenes in the movie *Interstellar*.

1. A quick open-source search for "Interstellar cornfield filming location town" reveals that the 500-acre cornfield was planted just outside the town of **Longview, Alberta, Canada**.
2. Next, searching for the geographic coordinates of Longview, Alberta yields **50.53°N, 114.23°W.**
3. Following the requested two-decimal format (`ab.cd,ef.gh`), we format the coordinates to fit the flag wrapper.

🚩 The Flag

Flag: `RAD{50.53,114.23}`

2. Robots

Points: 100

📝 Description

> In Interstellar, the robots TARS and CASE play a crucial role during the space mission aboard the spacecraft Endurance. Unlike many science-fiction films that rely heavily on CGI, director Christopher Nolan opted to build fully functional practical robot props for filming. The robot TARS was constructed as a physical prop that could be operated on set by puppeteers and mechanical rigs. Its distinctive rectangular monolith design allowed it to move in several unique configurations, including walking and rolling. These practical robots were built by a well-known Hollywood special effects company responsible for constructing physical props and mechanical effects used in numerous major films. Your task is to identify the special effects company that built the practical TARS robot used during filming. Flag Format: RAD{Location} Note: The first letter of all the words are Capital Letter s and the words have whitespace in between them

🔍 Solution

This is a straightforward OSINT challenge. Searching for the company responsible for building the physical robot props for *Interstellar* (or querying an AI assistant like Gemini) quickly provides the answer: **Legacy Effects**.

Formatting this into the requested flag wrapper with capitalized first letters and a space gives us the final flag.

🚩 The Flag

Flag: `RAD{Legacy Effects}`

STEGANOGRAPHY

1. Murph's First Signal

Points: 100

📝 Description

> NASA recovered an image from the bookshelf in Murph's room after strange gravitational disturbances were detected. Cooper believed gravity could be used to transmit simple messages across dimensions. The image appears normal, but analysts suspect a hidden signal embedded within the pixels. Can you recover the message Cooper tried to send?

🔍 Solution

The description mentions a hidden signal embedded within the pixels of an image, which points directly to image steganography.

To extract the hidden data, we can use `zsteg`, a highly effective tool for detecting hidden data in PNG and BMP files. By running the tool with the `-a` (all) flag, we force it to try all known methods of data extraction (like LSB steganography across various color channels).

```
zsteg -a bookself_signal.png
```



🚩 The Flag

`RAD{5T4Y_1N_51L3NC3!}`

2. What You Need

Points: 100

📝 Description

> A strange line was found in the Endurance system logs: You don't want anything. You already have what you want. Maybe it's just a message… or maybe it's a hint. Look carefully — the answer might already be in front of you.

🔍 Solution

At first glance, the text appears to be a normal log entry. However, the description hints that the answer might be "in front of you" and requires looking "carefully." This is a classic indicator of hidden text or invisible characters.

1. Dropping the provided text into a tool like **CyberChef** and viewing the raw output (or using the "To Hex" / "Show non-printing characters" recipe) reveals a massive block of invisible Unicode characters hidden between the standard letters.
2. The payload consists of various Zero-Width Characters (ZWCs), such as:
   - `\u200B` (Zero Width Space)
   - `\u200C` (Zero Width Non-Joiner)
   - `\u200D` (Zero Width Joiner)
   - `\uFEFF` (Zero Width No-Break Space)
   - `\u2060`, `\u2062`, `\u2063`, `\u2064`
3. I use StegZero.com tool online to get the flag.

🚩 The Flag

Flag: `RAD{G4rG4ntua_St4y_CuRi0us_14/03/26}`

3. Event Horizon Relay

Points: 200

📝 Description

> The Endurance captured a still as it skimmed the accretion disk. TARS claims the frame carries a hidden relay packet. Your task is to extract the secret message from the image.

🔍 Solution

The challenge title "Event Horizon Relay" and the hint about "skimming the accretion disk" suggest that data is hidden directly within the image pixels.

1. **Initial Analysis:** Standard forensic enumeration of the provided JPG file yields nothing.

  ```bash
exiftool "challenge.png" # No unusual metadata
binwalk -e "challenge.png" # No hidden zips or embedded files
```

```
2. Format Transformation: JPG is a lossy format, which makes pixel-perfect
steganography difficult to maintain. The hint about a "relay packet" and the
nature of the challenge leads to the discovery or requirement of a lossless
version of the data: challenge.png.
```

3. LSB Bit Plane Investigation: With a PNG file in hand, we can look for Least Significant Bit (LSB) steganography. Instead of manually clicking through planes in StegSolve, zsteg is the perfect tool for detecting hidden data across various bit planes and channel orderings.

zsteg challenge.png

4. Flag Extraction: Scanning the output reveals the hidden flag. The tool identifies that the data was intentionally hidden slightly deeper than usual to evade basic LSB scanners:
   - Location: b2,rgb,lsb,xy
   - Mechanism: Data was hidden in the 2nd bit plane (b2), interleaved across the Red, Green, and Blue channels, using standard XY coordinate sequence reading.

🚩 The Flag
`RAD{TESSERACT_RELAY_73C2}`

FORENSICS

1. Echo Signal

📝 Description

> While exploring the region near Gargantua, the Endurance spacecraft intercepted a strange transmission. The signal doesn’t appear to be a normal image and seems to be encoded in an unusual format.
   Mission control believes the transmission may hide important data. Analyze the signal and recover what was sent across the void.
🔍 Solution
When presented with audio that sounds like rhythmic, high-pitched scratching or static, Slow-Scan Television (SSTV) is a primary suspect.
1. Passing the provided WAV file through an SSTV decoding tool (such as MMSSTV on Windows, QSSTV on Linux, or even a mobile app like Robot36) reveals the hidden data.
2. The audio is a frequency-modulated image signal:
   * **Sync pulses** at 1200 Hz mark the start of each of the 241 scan lines (at 150ms intervals).
   * **Luminance (Y)** data is encoded between 1500 Hz (black) and 2300 Hz (white).
   * **Chrominance (Cb/Cr)** data follows, allowing the decoder to perform a YCbCr to RGB conversion.
3. Decoding the audio paints a cinematic scene: a figure standing between a farmhouse and a truck under a stormy sky.

````
4. The hidden message is embedded directly within the reconstructed image.
### 🚩 The Flag
`RAD{S1GN4L}`


## 2. Corrupted Signal from Gargantua

### 📝 Description
> We received a visual transmission from Gargantua, but the image file
appears to be heavily damaged. Standard image viewers refuse to open it due
to file corruption. Can you bypass the damage and recover the hidden signal?
### 🔍 Solution
When attempting to open or analyze the provided PNG file, it immediately
becomes apparent that the file is corrupted—likely due to broken headers,
modified magic bytes, or trailing data.
1. Instead of manually repairing the hex values of the PNG chunks (which can
be tedious), we can rely on a file carving tool like `foremost`. `foremost`
ignores the broken file structure and extracts valid files based purely on
their internal headers, footers, and data structures.
2. We run the tool directly against the corrupted image:

   ```bash
   foremost -i corrupted_signal.png
````

The tool successfully identifies the valid image data buried within the broken wrapper and extracts a new, clean PNG file into its output directory.
Opening this newly carved image reveals the flag written out in plain sight.



** 🚩 The Flag
`RAD{M4GIC_BY73_F1X}`

3. Gargantua Transmission
````

📝 Description

> A damaged transmission log was recovered from a probe near the black hole Gargantua. The QR code is partially torn and noisy, but still recoverable. Reconstruct or repair the QR to reveal the secret.

🔍 Solution

Initial Observation

Opening `challenge.png` reveals a QR code where the entire right side (~25%) is destroyed by a gray "spaghettification" effect. The top-left and bottom-left finder patterns are intact, but the **top-right finder pattern is completely missing** — this is critical since QR decoders rely on all three finder patterns to even detect a QR code.

A direct decode attempt with `cv2.QRCodeDetector()` confirms this — it fails immediately. (cv2 is OpenCV)

Determine QR Parameters

To reconstruct the QR, we need to know its **version** (which dictates size) and **module size** (pixels per QR cell).

By analyzing the top-left finder pattern's center cross-section, we observe the classic 1:1:3:1:1 ratio:

```
Horizontal: 13px, 14px, 41px, 14px, 14px  →  total 96px for 7 modules
Vertical:   13px, 14px, 41px, 14px, 14px  →  total 96px for 7 modules
```

- **Module size**: `96 / 7 ≈ 13.7 px`
- **QR origin**: pixel `(365, 375)`
- **Total QR height**: `670 px` → `670 / 13.7 ≈ 49 modules`
- **QR Version**: `(49 - 17) / 4 = 8` → **Version 8 (49×49)**

Extract the Module Matrix

Sample the center pixel of each module and classify:

- `< 100` brightness → **black** (1)
- `> 200` brightness → **white** (0)
- `100–200` (gray zone) → **unknown** (-1)

Result: **582 out of 2401 modules (24.2%) are unknown/damaged.**

Reconstruct Structural Patterns

We restore the QR's mandatory structural elements that fall within the damaged zone:

1. Top-right finder pattern (rows 0–6, cols 42–48):

```
X X X X X X X
X . . . . . X
X . X X X . X
X . X X X . X
X . X X X . X
X . . . . . X
X X X X X X X
```

2. White separators around the restored finder pattern (row 7 and col 41).

3. Timing patterns — alternating black/white lines:

- Horizontal: row 6, cols 8–40
- Vertical: col 6, rows 8–40

4. Alignment patterns — Version 8 has alignment centers at positions `{6, 24, 42}`. We place the standard 5×5 alignment pattern at each grid intersection that doesn't overlap a finder
pattern: `(6,24)`, `(6,42)`, `(24,6)`, `(24,24)`, `(24,42)`, `(42,6)`, `(42,24)`, `(42,42)`.

5. Dark module at position `(4×8 + 9, 8) = (41, 8)`.

Step 5: Decode

With structural patterns restored, the remaining unknown data modules are handled by **QR error correction** (Reed-Solomon codes). Even with ~20% of data modules still missing, the error correction level built into the QR is sufficient to recover the full payload.

```
detector = cv2.QRCodeDetector()
data, bbox, rectified = detector.detectAndDecode(repaired_qr)
```

🚩 The Flag

`RAD{7h3_73553r4c7_516n4l_7r4n5m15510n_fr0m_64r64n7u4_r3c0n57ruc73d}`

4. Timestamp Whisper

📝 Description

> A deep-space probe relays telemetry through a ground station. Operators report that normal network traffic looks fine, but command acknowledgments are missing. Intel suggests a covert channel hidden in timing jitter rather than payloads.
> Your task is to analyze the capture and recover the hidden transmission.

🔍 Solution

Initial Observation

Opening the pcap in Wireshark, we find **390 packets** broken down as:

Protocol	Count	Purpose
UDP/DNS	320	Regular DNS queries to space-themed domains
ICMP	27	Echo requests (no replies)
UDP/NTP	24	NTP client packets (all fields zeroed)
TCP	19	SYN packets (no handshake completed)

The DNS traffic stands out: 320 queries cycling through 15 domains (`api.nasa.gov` , `interstellar.probe` , `telemetry.node` , etc.) at an extremely regular **50ms interval**. The ICMP, TCP, and NTP packets are scattered at irregular times and carry no meaningful payload — they appear to be decoy noise.

The challenge says the covert channel is hidden in **timing jitter**, not in payloads. So the data must be embedded in the timestamps themselves.

Identifying the Covert Channel

Measuring each DNS packet's timestamp against a perfect 50ms grid reveals something interesting. Taking the first DNS packet as the reference point (`t0 = 1700000000.099992`), every subsequent packet should arrive at `t0 + i * 0.050` seconds.

The actual jitter (deviation from the grid) for each packet is one of exactly **four values**:

Jitter (microseconds)	Dibit Value
0	0 (`00`)
1	1 (`01`)
10	2 (`10`)
11	3 (`11`)

Only these four values ever appear — no other jitter. This is clearly intentional: each DNS packet encodes a **2-bit symbol** (a dibit) via its microsecond-level timing offset.

Extracting the Hidden Data

With 320 packets each carrying a dibit, we have 320 two-bit values. But not all bits carry data.

Separating each dibit into its **upper bit** (bit 1) and **lower bit** (bit 0):

- **Upper bits** produce a repeating pattern: `10000100001000010...` — this is just camouflage/noise.
- **Lower bits** (the LSBs) produce meaningful data.

Extraction logic:

```
Jitter 0  -> dibit 0 (binary 00) -> LSB = 0
Jitter 1  -> dibit 1 (binary 01) -> LSB = 1
Jitter 10 -> dibit 2 (binary 10) -> LSB = 0
Jitter 11 -> dibit 3 (binary 11) -> LSB = 1
```

This gives us **320 bits** from the LSBs.

Decoding to ASCII

Group the 320 bits into **8-bit chunks** (MSB-first) and convert each byte to ASCII:

```
Packet dibits:  0  1  0  3  0  0  1  0 | 2  1  0  0  0  2  0  1 | ...
LSBs:           0  1  0  1  0  0  1  0 | 0  1  0  0  0  0  0  1 | ...
                ---------------------   ---------------------

                0x52 = 'R'                0x41 = 'A'
```

320 bits / 8 = **40 ASCII characters**.

🚩 The Flag

`RAD{1N73R574R_73L3M37RY_5T4T10N_0RB1T9X}`

5. Murphy's Echo

📝 Description

> This challenge involves analyzing a network capture (`interstellar_relay.pcap`) to find a flag hidden within telemetry traffic. While the file contains standard HTTP noise, the core of the problem lies in the repeated communication between a local source and a host named "tesseract relay".

🔍 Solution

This challenge tests your ability to filter out noise and identify covert channels within network traffic.

1. **Traffic Analysis:** Inspecting the PCAP reveals two distinct groups of traffic. There is decoy traffic consisting of standard HTTP GET requests and DNS lookups (e.g., nasa.gov, github.com), and suspicious traffic consisting of custom telemetry and status packets sent to the "tesseract relay."

2. **Identifying the Covert Channel:** The status packets (`status1` through `status4`) simply return "OK" to act as a heartbeat and make the connection look legitimate. However, a closer look at the telemetry packets reveals unique strings of alphanumeric characters appended to the end of the payloads. The specific character set (uppercase A-Z and digits 2-7) is a dead giveaway for **Base32** encoding.
3. **Extraction & Reconstruction:** We extract the hidden payload fragments from the six telemetry packets in sequential order:

`telemetry1 : KJAUI6ZRNY3T`
`telemetry2 : G4RVG42DCMJT`
`telemetry3 : OJPTCNK7GRRD`
`telemetry4 : A5JXL5WDA5RT`
`telemetry5 : L42G4ZC7G4YW`
`telemetry6 : 2M35`

4. **Decoding:** Concatenating these fragments gives us the full encoded string: `KJAUI6ZRNY3TG4RVG42DCMJTOJPTCNK7GRRDA5JXL5WDA5RTL42G4ZC7G4YW2M35` . Decoding this from Base32 yields the hidden flag.

(Here the challenge author effectively combined data fragmentation, protocol tunneling, and Base32 encoding to bypass simple pattern-matching detection!)

🚩 The Flag

Flag: `RAD{1n73r574113r_15_4b0u7_l0v3_4nd_71m3}`

6. Event Horizon

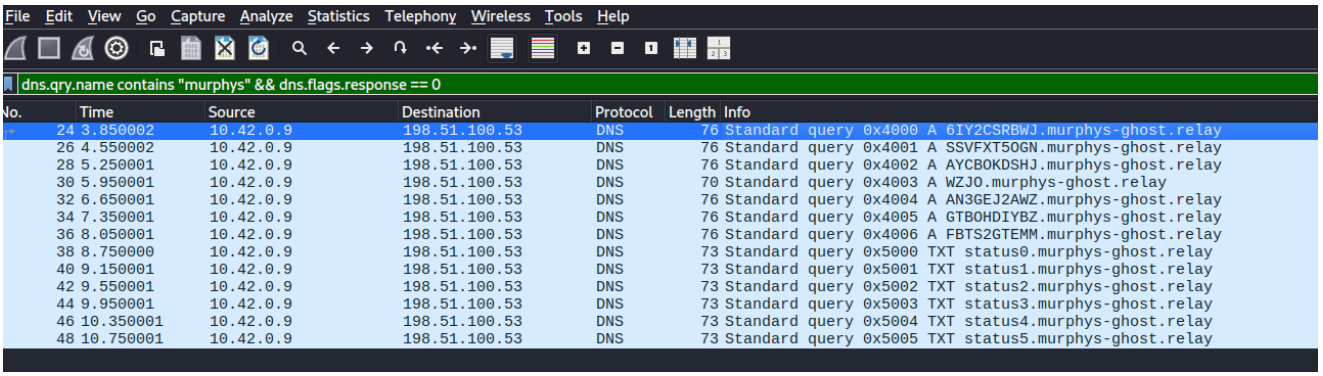
📝 Description

> During a Tesseract telemetry experiment, covert data was smuggled inside otherwise ordinary DNS traffic to a deep-space relay. The objective is to reconstruct the hidden message and recover the flag.

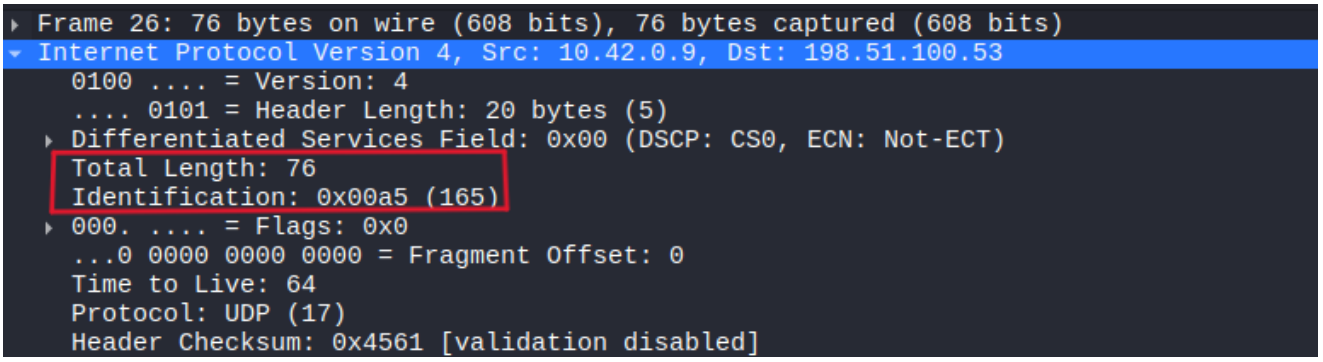
🔍 Solution

This challenge involves deep packet inspection, identifying out-of-order data exfiltration, and breaking multiple layers of obfuscation and encryption.

1. **Initial Analysis in Wireshark:** Opening the PCAP, we observe a high volume of DNS traffic. We isolate the relevant traffic using the display filter:
`dns.qry.name contains "murphys" && dns.flags.response == 0`
The traffic reveals "Status Checks" and "Data Exfiltration" via long, alphanumeric subdomains (e.g., `6IY2CSRBWJ`). The character set implies Base32 encoding.



1. **Identifying the Anomaly:** Extracting the subdomains chronologically yields binary noise. Noticeably, one chunk, `WZJO`, is only 4 characters long. In Base32, a padded/shortened chunk should appear at the very end of the sequence, indicating the packets arrived out of order or were intentionally scrambled by the attacker to evade basic static analysis.
2. **The Breakthrough (Sequence Numbers):** If the chunks are scrambled, the exfiltration tool must have embedded a sequence number somewhere in the packet headers so the receiving server could reassemble the payload. By carefully inspecting the packet details, we find sequential hex values (e.g., `a0`, `a1`, `a2`, etc.) tied to the specific subdomain chunks. This allows us to manually extract and sort the chunks into their correct sequence:



- **Index 0 (a0):** `AYCBOKDSHJ`
- **Index 1 (a1):** `GTBOHDIYBZ`
- **Index 2 (a2):** `AN3GEJ2AWZ`
- **Index 3 (a3):** `6IY2CSRBWJ`
- **Index 4 (a4):** `FBTS2GTEMM`
- **Index 5 (a5):** `SSVFXT5OGN`
- **Index 6 (a6):** `WZJO`

4. **Breaking the Payload Obfuscation:** Even in the correct order, direct Base32 decoding still fails, meaning the chunks themselves are obfuscated. In CTF cryptography, attackers frequently apply transformations to alternating elements to break automated tools. Testing this hypothesis, we apply a string reversal (`[::-1]`) specifically to the odd-indexed chunks (`1` , `3` , and `5`).
5. **Decryption Script:** With the logic mapped out, we can automate the obfuscation reversal, Base32 decoding, and the final thematic XOR decryption (using the uppercase

key `TESSERACT`) with a Python script:

```
import base64

# 1. Ordered chunks extracted from the packet headers
chunks = [
    "AYCBOKDSHJ", # a0 (Even)
    "GTBOHDIYBZ", # a1 (Odd)
    "AN3GEJ2AWZ", # a2 (Even)
    "6IY2CSRBWJ", # a3 (Odd)
    "FBTS2GTEMM", # a4 (Even)
    "SSVFXT5OGN", # a5 (Odd)
    "WZJO"        # a6 (Even)
]

# 2. Reverse the odd-indexed chunks to undo the interleaved obfuscation
chunks[1] = chunks[1][::-1]
chunks[3] = chunks[3][::-1]
chunks[5] = chunks[5][::-1]

# 3. Reassemble and Base32 Decode
final_b32 = "".join(chunks)
raw_bytes = base64.b32decode(final_b32)

# 4. XOR Decrypt using the thematic uppercase key
key = b"TESSERACT"
xored = bytearray([raw_bytes[i] ^ key[i % len(key)] for i in
range(len(raw_bytes))])

print(f"FLAG: {xored.decode(errors='ignore')}")
```

Running the script successfully undoes the array obfuscation, decodes the Base32, and XOR decrypts the final payload.

🚩 The Flag

Flag: `RAD{7h3_t3553r4c7_1s_7h3_k3y_70_574y1n6}`

REVERSE

1. Singularity Protocol

📝 Description

> You are provided with a binary named `singularity_protocol`. Upon execution, it displays a new equation every session (SESSION, K[0..4], SBOX_KEY, CIPHER). The

> binary employs anti-debugging techniques and compares your input against the generated cipher. Can you cross the event horizon?

🔍 Solution

The challenge presents a binary that generates a "new equation" (cipher) every session, making it seem like you need to break the dynamic encryption. However, this is a misdirection.

1. **Reconnaissance:** Running the binary reveals its output format. A quick analysis shows it uses `ptrace` for anti-debugging and `memcmp` to verify the final input.
2. **Dynamic Bypass (LD_PRELOAD):** To bypass the anti-debug and inspect the memory, you can build a quick LD_PRELOAD shim with two hooks:
   Hook `ptrace()` to return `0` (bypassing the anti-debug check).
   Hook `memcmp(buf1, buf2, 30)` to dump both buffers and return `0` (bypassing the check and capturing internals).
3. **Static Analysis:** Disassembling the binary with `objdump -d` reveals the true vulnerability. The "secret plaintext" passed to the encrypt function comes from two static `.rodata` constants, not the dynamic session key. The session randomness only affects the outer cipher wrapping, leaving the underlying secret static.
4. **Extracting the Keys:** Extracting the static XOR keys from the `.rodata` section:
   `0x3058–0x305b` = `1a 2b 3c aa` → XORed together = `0xa7` (Key for flag bytes 0–15)
   `0x3054–0x3057` = `4d 5e 6f cf` → XORed together = `0xb3` (Key for flag bytes 16–29)
5. **Decryption:** With the static keys recovered, we can decrypt the flag locally using a quick Python script:

```
# Decrypting the static rodata payload

plain = bytearray(30)

plain[0:16]  = bytes([b ^ 0xa7 for b in rodata_0x3600])  # Yields:
"RAD{Gr4v1ty_Br34"

plain[16:30] = bytes([b ^ 0xb3 for b in rodata_0x3060])  # Yields:
"k5_4ll_rul35!}"

print(plain.decode())
```

Verification: Passing the decrypted flag back into the binary confirms the solution.

```
echo -n "RAD{Gr4v1ty_Br34k5_4ll_rul35!}" | ./singularity_protocol
```

**Output: SINGULARITY SOLVED — You crossed the horizon!

🚩 The Flag

`RAD{Gr4v1ty_Br34k5_4ll_rul35!}`

2. Tesseract Relay

📝 Description

> A binary that checks for a debugger, reads a 36-character Morse transmission, and validates the Morse structure using a lookup table. If the correct conditions are met, it computes a hash and reconstructs the flag. Can you bypass the checks and recover the transmission?

🔍 Solution

This challenge involves reversing a Linux binary to understand its anti-debugging mechanisms and flag reconstruction logic, then writing a custom script to bypass the dynamic checks entirely.

1. **Reverse the Binary:** Opening the binary in Ghidra and analyzing the main function (`FUN_001011c0`) reveals the core flow:

It checks for a debugger using `/proc/self/status` .
It reads a 36-character Morse transmission.
It validates the Morse structure using a lookup table (`DAT_001025e0` *).*
It computes a hash and checks if `bVar14 == 0xea` . If satisfied, the program reconstructs the flag.

2. **Locate Flag Reconstruction:** The flag isn't stored in plain text. Instead, it is built dynamically using three encrypted arrays located at `DAT_00102668` , `DAT_00102658` , and `DAT_00102648` . These arrays are decoded using XOR operations with incrementing keys.
3. **Extract Arrays:** We can extract the hex values of these arrays directly from Ghidra:

`arr1 = [FA, E4, F2, F8, D1, 59, 08, 04, 73, 02, 7A]`
`arr2 = [88, A7, F6, 4E, 4B, 49, 4D, 53, 10, 5E, 01]`
* `arr3 = [43, D9, 8A, BF, F0, A4, 91, DB, FB, FF]`

1. **Recreate Flag Logic & Brute Force:** Instead of fighting the Morse code validation and anti-debugging checks dynamically, we can just reimplement the flag reconstruction logic in Python. Since one of the XOR variables (`bVar17`) isn't immediately obvious, we can just brute-force all 256 possible byte values.

```
arr1 = [0xFA, 0xE4, 0xF2, 0xF8, 0xD1, 0x59, 0x08, 0x04, 0x73, 0x02, 0x7A]
arr2 = [0x88, 0xA7, 0xF6, 0x4E, 0x4B, 0x49, 0x4D, 0x53, 0x10, 0x5E, 0x01]
arr3 = [0x43, 0xD9, 0x8A, 0xBF, 0xF0, 0xA4, 0x91, 0xDB, 0xFB, 0xFF]
def build_flag(v):
```

```
    out = []
    b = 0x42
    for x in arr1:
        t = x ^ b
        b = (b + 0x0d) & 0xff
        out.append(t ^ 0xea)
    b = 0x5e
    for x in arr2:
        t = x ^ b
        b = (b + 0x0d) & 0xff
        out.append(t ^ v)
    b = 0x7a
    for x in arr3:
        t = x ^ b
        b = (b + 0x0d) & 0xff
        out.append(t ^ 0x6d)
    return bytes(out)
# Brute-force the unknown variable (bVar17)
for i in range(256):
    s = build_flag(i).decode(errors="ignore")
    if s.startswith("RAD{") and s.endswith("}"):
        print(s)
```

5. Result: Running the script instantly brute-forces the missing byte and outputs the recovered flag.

🚩 The Flag

`RAD{M0rs3_Thr0ugh_Th3_T3ss3r4ct}`

MISC

1. Fragments Beyond Gargantua

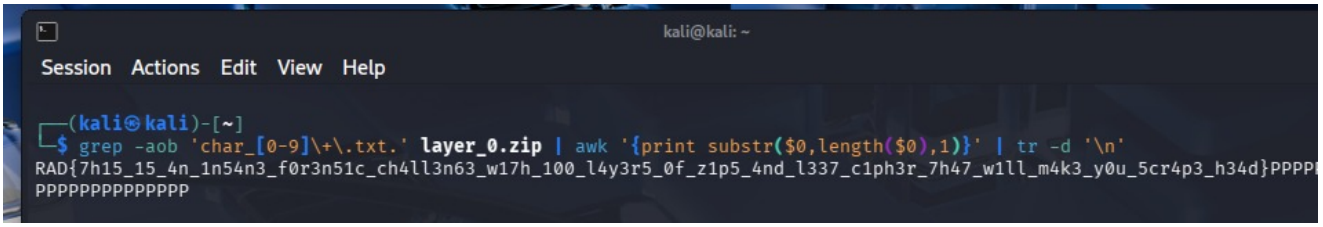
📝 Description

> The Endurance spacecraft intercepted a strange transmission near Gargantua. The signal appears to be fragmented across multiple compression layers, as if someone tried to hide it deep within recursive archives. Each layer reveals only a single character of the message.

🔍 Solution

The downloaded file is a recursive archive (often called a zip bomb) consisting of 112 nested `.zip` files.

1. Manually extracting 112 layers is tedious, so the quickest way to solve this is by writing a short bash or Python script to automate the extraction process.
2. Below shell command extracts exactly one character per layer and prints the flag.




```
grep -aob 'char_[0-9]\+\.txt.' layer_0.zip | awk '{print
substr($0,length($0),1)}' | tr -d '\n'
```

🚩 The Flag

Flag:
`RAD{7h15_15_4n_1n54n3_f0r3n51c_ch4ll3n63_w17h_100_l4y3r5_0f_z1p5_4nd_l337_c1ph3r_7h47_w1ll_m4k3_y0u_5cr4p3_h34d}`